Week 9 Example Problems: Testing / QA

Authors: Liliana Hotsko, Alina Lytovchenko, Sofiia Tkach, Zesheng(Ethan) Jia

Instructions

GitHub Repository: https://github.com/sonyaaat/CS846-Week9-Testing

1. Example Problems

Problem A: Checkout Service Testing

Use the GPT-4 model for this problem.

Task Description: You are testing the checkout system of an online store (CheckoutService.process_checkout). This service calculates the final order totals by applying several rules in order: item-level discounts, coupon codes (SAVE10, SUMMER20, FLASH5), loyalty point redemptions, shipping fees, taxes, and payment processing. Each rule can interact, which can lead to small bugs, especially around thresholds and compatibility.

Your task is to write pytest unit tests that identify these bugs. Your tests should thoroughly cover each individual rule and include pipeline integration tests to ensure consistent results across steps. Follow the public contract provided, not just the current behavior of the code, and suggest additional edge cases that are not explicitly listed.

Starter Code: See `checkout_service.py` in the `problem_A` folder.

Required Workflow:

- Summarize expected checkout behavior.
- Propose at least 5 extra edge cases beyond those listed.
- Identify implementation mismatches.
- Write final pytest tests.

Problem B

> All tasks should be completed in Python using pytest. Use the GPT-4 model for this problem set. For testing, use the `@pytest.mark.parametrize` decorator to efficiently check multiple input cases and uncover bugs in the functions.

Problem B_1: User Validation Testing

Task Description:
You are given a user validation module with functions that validate email addresses, ages, usernames and passwords. Your task is to write comprehensive unit tests using pytest that will reveal bugs in the validation logic.

Starter Code:
See `user_validator.py` in the ProblemB/Problem_B_1 folder.

Problem B_2: Order Processing Decomposition (Guideline 4 Focus)

Task Description:
You are given a complex `process_order()` method that handles validation, calculations, discounts, taxes, payment processing, and confirmation generation all in one function. Your task is to write comprehensive tests that expose the hidden bugs in this complex method.

Starter Code:
See `order_processor.py` in the ProblemB/Problem_B_2 folder.

Problem B_3: Data Parser Edge Cases (Combined Guidelines 3 & 4)

Task Description:
You are given a data parsing module with functions for CSV parsing, JSON configuration validation, number extraction, and data type validation. Each function has multiple edge cases that need thorough testing.

Starter Code:
See `data_parser.py` in the ProblemB/Problem_B_3 folder.

Expected Discovery: The combination of both guidelines will reveal numerous edge case bugs and parsing failures that simple testing approaches miss.

Problem C: Reach target test coverage

Use the GPT-5-mini model for this problem.

Generate tests for FLASK `app.py` file and use `eval_test_coverage.py` to measure how well LLM-generated tests cover the functionality in it.

Target source file: `ProblemC/flask/src/flask/app.py` (435 executable lines) **Coverage goal:** ≥ 60%

Setup

```
pip3 install pytest pytest-cov flask
cd problem_C
git clone https://github.com/pallets/flask.git
```

STEP 1: Write the prompt and generate the tests

Place the test file into the problem_C folder root and name it "test_flask1.py".

STEP 2: Evaluate a test file

```
python3 eval_test_coverage.py \
  --source-file flask/src/flask/app.py \
  --test-file "test_flask1.py" \
  --work-dir flask
```

STEP 3: Review the Guideline

STEP 4: Rewrite the prompt and generate the tests

Place the test file into the problem_C folder root and name it "test_flask2.py".

STEP 5: Compare 2 test files and their test coverage

```
python3 compare_coverage.py --src flask/src/flask/app.py \
  --unguided "test_flask1.py" \
  --guided "test_flask2.py" \
  --work-dir flask
```

Problem D

Setup (pytest): Before starting, install `pytest` with `python3 -m pip install -r ProblemD/requirements.txt` (or `python3 -m pip install pytest`).

Problem D_1: Understand the Mini Autograd API and Generate Baseline Tests

Task Description:
This problem uses a **pure-Python mini autograd engine** (`ProblemD/student/src/mini_autograd.py`) that is inspired by PyTorch autograd, but does not require installing PyTorch.
In this context, an **autograd function** means a custom operation with an explicit forward computation and backward (gradient) rule. In `ProblemD`, custom operations are implemented using a `Function.apply(...)` API similar to `torch.autograd.Function`.

Use an GPT-5-mini to generate a first pytest test files that checks baseline behavior for the mini engine: Review and refine the generated tests so they are clean and runnable. Run your tests for detecting bugs, but do not change the implementation code yet.

Starter Code:
`ProblemD/student/src/mini_autograd.py`
`ProblemD/student/src/demo_custom_functions.py`
`ProblemD/student/tests/test_examples_smoke.py` (style example only)

Problem D_2: Design Edge-Case Tests for Autograd Graph Semantics

Task Description:
Use your LLM to extend the pytest suite and test **graph semantics**, not just arithmetic outputs. Add tests

for behavior such as: gradient accumulation through shared subgraphs, etc.

Your tests should be small and deterministic. Avoid hidden dependencies and avoid rewriting the starter code in the test file.

Starter Code:
`ProblemD/student/src/mini_autograd.py`
`ProblemD/student/src/demo_custom_functions.py`

Problem D_3: Use an LLM to Generate Tests for the Standardized `axpy` Function

Task Description:
All students will use an LLM to generate pytest tests for the same function: `axpy(a, x, y)` in `ProblemD/student/src/demo_custom_functions.py`.
Your goal is to produce a clean, runnable test file for `axpy` and improve it through prompt refinement.

You must document and submit:

1. the model used,
2. the prompt(s) used,
3. the final generated/refined test code, and

Starter Code:
`ProblemD/student/src/demo_custom_functions.py`
`ProblemD/student/src/mini_autograd.py`

Problem D_4: Bug-Fix Challenge Using Your Generated Tests

Task Description:
Use the tests you generated/refined in Problems D_1–D_3 to identify and fix bugs in the starter implementation (you may use an LLM to propose patches, but you must verify them with tests):

- `ProblemD/student/src/mini_autograd.py`
- `ProblemD/student/src/demo_custom_functions.py`

Requirements:

1. Run your own tests first and use failures to guide debugging.
2. Fix implementation bugs without deleting features.
3. Keep your tests (and any LLM-generated tests) as evidence of how you found the bugs.
4. Make sure your pass all your tests.

Starter Code:
`ProblemD/student/src/mini_autograd.py`
`ProblemD/student/src/demo_custom_functions.py`